

PyQCD 仓库分析与梯度流重整化胶子 TMD-PDF 物理链

opencode analy

2026-08-18

摘要

本报告对主仓库 `/root/PyQCD` (PyQCD By ZhangXin, 格点 QCD 研究仓库) 做全库只读分析, 解析其如何组织代码以完成核心目标——[计算使用梯度流重整化方案的核子中的胶子 TMD-PDF](#)。分析范围覆盖主库自有代码 (pyqed 包 91 个 `.py`、examples、docs 53 篇 tex、logs 测试套件, 总计排除 refer 后 252 个文件), refer 作为只读参考知识库引入但不深剖。核心结论: 仓库以「蒸馏管线 + 胶子 OPE + 梯度流重整化 + 混合/Z_R + NLO 匹配 + 连续极限外推」为主线, 采用多后端 (numpy/cupy/torch) + h5 IO + MPI 并行的工程骨架; 一致性验证 [AGENTS.md:11](#) 显示与 docker 基线 A-E 全 0 差异、全量 237/237 PASS。三视角: 结构 (入口/核心/工具/文档/参考分层)、关系 (vertex→2pt→ope→3pt→4pt→analysis 数据依赖链)、思路 (照抄成功实例 docker-v20260805 与 refer 逻辑、自包含实现、可回放复现)。

目录

1 仓库概览	2
1.1 目录结构	2
1.2 规模统计与 git 信息	3
2 主体结构	3
3 各部分关系	4
3.1 数据依赖主链	4
3.2 工程支撑关系	4
4 项目思路	4
4.1 设计意图	4
4.2 演进脉络	5
4.3 典型工作流	5
5 主题分析: 梯度流重整化胶子 TMD-PDF 计算链	5
5.1 A1 目的与前期准备	5
5.2 A2 输入	5
5.3 A3 输出 (应是什么)	5
5.4 A4 整体框架	6

5.5	A5 关键细节	6
5.6	A6 任务如何正确执行	6
5.7	A7 实际执行情况	6
5.8	A8 实际输出情况	6
5.9	A9 结果分析	6
5.10	A10 综合分析	6
5.11	A11 结尾总结	7
5.12	A12 结尾评价	7
5.13	A13 补充说明	7
5.14	A14 参考源	7
5.15	A15 附录与附件	7
6	关键代码/文档片段	7
6.1	Wilson flow 单步 (RK3)	7
6.2	胶子 TMD 算符 O 组合	7
6.3	Clover 场强四叶构造 (节选)	8
6.4	NLO 匹配核 $g_0..g_3$ (按 ξ 分区)	8
6.5	匹配 Z_{ij} 矩阵结构	9
7	参考源清单	10
8	结论与局限	10

1 仓库概览

1.1 目录结构

仓库顶层以 `pyqcd/` 主包为核心, `examples/` 提供成功实例基线与规范测试, `docs/` 存放中文 LaTeX 笔记与报告, `logs/` 为各功能测试套件, `refer/` 为只读参考库。

```

1 PyQCD/ (main repo /root/PyQCD, git: main @ 9f74beb)
2 +-- AGENTS.md README.md LICENSE env.sh
3 +-- pyqcd/          main package (91 .py excluding refer)
4   +-- lattice/      constants / gamma,sigma (DR basis)
5   +-- tools/        backend numpy/cupy/torch + h5 I/O + einsum cache
6   +-- vertex/       VdV/VVV vertices + phase factors
7   +-- contraction/  auto-Wick / baryon operator / seqperam
8   +-- operator/     Clover Fmunu / dual Ftil / gluon OPE / TMD staple
9   +-- analysis/     jackknife/bootstrap/meff/ratio/disconnected/dispersion
10  +-- renorm/        * gradient flow / Z_R / hybrid / NLO match / extrap / TMD
11  +-- smear/        HYP smearing (flow alternative)
12  +-- pipeline/      9-step orchestration + tmd step
13  +-- testing/       integration tests
14  +-- parallel/      MPI planning + meta-task scheduling
15 +-- examples/       docker-v20260805 baseline + pyqcd demos/tests + test0
16 +-- docs/           53 Chinese LaTeX notes + analy/pure reports

```

```

17 +-- logs/                test0*/stab1/test6/test7 functional suites
18 +-- cpp/                C++ backend placeholder
19 +-- refer/              read-only references (zengch/donghx/.../git-rep/*)
20 +-- data/ output/ plots/ test_out/  runtime data / products / figures

```

1.2 规模统计与 git 信息

指标	实测值
主库文件总数（排除 refer/___pycache___）	252
Python 文件（排除 refer）	91
Markdown 文件（排除 refer）	71
LaTeX 文件（排除 refer）	64
Shell 脚本（排除 refer）	18
docs 内 tex 篇数	53
全库文件总数（含 refer）	9467
当前分支 / HEAD	main / 9f74beb (2026-08-17)
最新标签	test8、stab4（版本序）

表 1: 仓库实测统计（数据来源: find/wc/git 实测，2026-08-18）

注：全库 9467 文件含 **refer/git-rep/*** 外来软件库（含 EasyDistillation、LQCD_Master、PyQUda、lamet-agent、quda），按约定仅作为参考知识库引入，不纳入主库主体分析（见第 3 节）。

2 主体结构

仓库采用「子包 + _ 前缀私有模块 + 显式 re-export」架构（[确证](#)，pyqcd/AGENTS.md 子包表）。各层职责如下。

层次	目录	职责
入口层	examples/	成功实例基线 docker-v20260805 + pyqcd 规范示例/测试 + test0 蒸馏一致性套件
核心层	pyqcd/renorm/	* 梯度流/z_R/混合/NLO 匹配/外推/TMD 提取（理论核心）
核心层	pyqcd/operator/	Clover 场强 F、对偶 F、胶子 OPE 算符、TMD staple 扩展
核心层	pyqcd/analysis/	Jackknife/Bootstrap/meff/ratio/disconnected/色散拟合
核心层	pyqcd/pipeline/	9 步管线集中配置与调度（+tmd 步）
核心层	pyqcd/vertex/ contraction/	VdV/VVV 顶点、自动 Wick、重子算符、seqperam

层次	目录	职责
工具层	pyqcd/tools/	后端切换 numpy/cupy/torch、缓存 einsum、切片、h5 IO
工具层	pyqcd/parallel/	MPI 并行规划与元任务调度
工具层	pyqcd/lattice/ smear/	常数/ γ/σ 矩阵 (DR 基); HYP 涂抹 (梯度流备选)
测试层	pyqcd/testing/	集成测试函数 (conftest 入口)
文档层	docs/	52 篇中文 LaTeX 笔记 + analy/pure 报告
参考层	refer/	只读参考 (zengch/donghx/huangcl/.../git-rep/*), 逻辑照抄不 import
产物层	logs/ output/ plots/	功能测试套件、运行数据、图表产物

表 2: 主体结构分层 (数据来源: 全库索引与 pyqcd/AGENTS.md 实测)

3 各部分关系

3.1 数据依赖主链

物理计算链以蒸馏管线串联：规范场经 Wilson flow 涂抹后，构造胶子 TMD 算符 O，经自重整化/混合得到 hR，傅里叶变换后经 NLO 匹配得光锥 PDF，最后连续极限外推。

要点

依赖主链: gauge U $\rightarrow$ wilson_flow(`_gradient_flow.py`) $\rightarrow$ gluon_tmd_operator(`_tmd.py`) $\rightarrow$ self_renormalized_ratio / hR_lambda(`_zr.py, _hybrid.py`) $\rightarrow$ hR_PDF(`_matching.py`) $\rightarrow$ extrapolate(`_extrapolate.py`)。各环节参考源附证据色标注，下游模块只依赖上游输出数组，解耦清晰。

3.2 工程支撑关系

多后端统一经 pyqcd/tools 的 `get_backend()` 适配层访问；管线各 step (pyqcd/pipeline/_steps.py:275,418,546,665,758,841) 通过 `save_array/_load_any` (h5 优先) 读写中间产物，使 numpy/cupy/torch 三后端可替换且不污染算法代码。MPI 并行 (pyqcd/parallel) 仅调度 step $\times$ conf 元任务，分析作图层仅 rank 0 执行。

4 项目思路

4.1 设计意图

仓库的根本动机是把「docker-v20260805 成功实例」的可复现物理结果，以自包含、可测试、多后端可移植的方式重写成 pyqcd 主包 (推断, 由 AGENTS.md「照抄 docker 逻辑自包含」「不 import refer」

反模式条目推出)。关键设计选择：(1) 张量布局统一 ($Nt, Nz, Ny, Nx, 4, 3, 3$) 维序 t, z, y, x ；(2) 禁止直接 import cupy，统一经后端适配层；(3) 中间产物一律 h5 读写，旧 .npy/.npz 回退。

4.2 演进脉络

git 历史显示近期以 tag **stab1** (全功能真实数据实战) → **test7** (服务器 100 组态正式版) → **stab4/test8** 推进；功能测试套件 (test0*/stab1/test6) 逐层沉淀，表明工程重心从「单点复现」走向「规模化并行实战」。torch 后端为后期新增 (实测 CPU 梯度流 2.7–4.8x 加速)。

4.3 典型 workflow

一次典型任务：pythonexamples/pyqcd/conftest.py 跑全量 18 项单测；或 bashexamples/test0/run-local.sh 蒸馏管线一致性全量复现；规模化则 python-mpyqcd.parallel--confs... 经 MPI 调度。

项目思路

本仓库「先有成功实例与参考，再自包含重写并套测试保护网」的组织方式，使其既保物理结论可复现 (A-E 全 0 差异)，又具备多后端/并行扩展能力——这是它「长这样」的灵魂原因。

5 主题分析：梯度流重整化胶子 TMD-PDF 计算链

本节以「一项具体工作」的 15 步框架 (A 代码工作框架) 完整重现该计算链。判定：对象以代码/数值流程为主，故主体用 A 框架，物理元素并入相应步骤。

5.1 A1 目的与前期准备

为什么做：在格点上以梯度流重整化方案计算核子胶子 TMD-PDF——裸矩阵元 (蒸馏 2pt + 胶子 OPE staple 算符) → Wilson flow 涂抹 → 混合/Z_R → λ 外推 → 傅里叶 → NLO 匹配 → 连续极限外推 (**AGENTS.md**：核心目标)。前置：numpy/cupy/torch 后端、规范场 gauge 张量、蒸馏源/汇传播子。

5.2 A2 输入

数据/参数：规范场 $U(Nt, Nz, Ny, Nx, 4, 3, 3)$ ；纵向位移 z 、横向位移 $b_\perp$ 网格；流时间 τ (物理约定 $\tau = 3a^2$, NieMiera 2025)；匹配标度 $\mu = 2 \text{ GeV}$ ；截断 $\lambda_s = 0.3 \text{ fm}$ (**pyqcd/AGENTS.md**：关键约定)。

5.3 A3 输出 (应是什么)

梯度流重整化的矩阵元 $O(z, b_\perp) \rightarrow$ 自重整化比值 $R \rightarrow hR(z) \rightarrow$ 光锥 PDF $x \cdot g(x, b_\perp) \rightarrow$ 连续极限外推后的物理 TMD-PDF；并产出中间 h5 与图表、LaTeX 报告。

5.4 A4 整体框架

模块组织: `renorm/_gradient_flow.py` (流)、`operator/_gluon_ope.py` + `renorm/_tmd.py` (算符与矩阵元)、`renorm/_zr.py`、`renorm/_hybrid.py` (重整化)、`renorm/_matching.py`、`renorm/_tmdextract.py` (匹配)、`renorm/_extrapolate.py` (外推)。主流程由 `pipeline/_steps.py:run_pipeline` (`pyqcd/pipeline/_steps.py:1418`) 调度 9 步。

5.5 A5 关键细节

关键算法: Wilson flow 用 RK3 (Luescher 2010); Clover F 四叶平均; TMD 算符 O 的可乘性组合 $O = M^{tx;tx} + M^{ty;ty} - 2M^{xy;xy}$; 匹配核 `g_0..g_3` 按 $\xi = x/y$ 分区; Z_{ij} 矩阵结构为

$$Z_{ij} = \delta_{ij} + \frac{\alpha_s C_A}{2\pi} dx \frac{x}{|x|} \times \left[\frac{g_{ij}}{y_j} - \delta_{ij} y_j \sum_k \frac{g_{ik}}{y_k^2} \right]. \quad (1)$$

易错点: 色迹维度、roll 轴序、复数 dtype 一致性——由后端适配层统一保证。

5.6 A6 任务如何正确执行

步骤/命令: `pythonexamples/pyqcd/conftest.py` (单测链验证) ; `pythonexamples/pyqcd/verify_consistency.py` (A-E 一致性)。

5.7 A7 实际执行情况

蒸馏管线一致性测试: `bashexamples/test0/run-local.sh` 复现 docker 基线全量 9 步; 一致性验证 `verify_consistency.py` 对 A-E 五组对照全部 0 差异 (`AGENTS.md:11`)。

5.8 A8 实际输出情况

已验证结论: `conf6250` 中间数据逐位一致 (`rel=0.000e+00`), 全量 237/237 PASS (`AGENTS.md:70-78`); `stab1` 实战 10 组态、45 项断言、106 图 + 报告。

5.9 A9 结果分析

结合图表与日志: P_0 m_{eff} 平台 ≈ 1.12 GeV (质子质量, 已验证)、 $P_2 \approx 1.56$ GeV (色散)、 E_0 与 m_{eff} 平台一致 (`AGENTS.md:105-110`)。torch 后端 CPU 梯度流 2.7–4.8x 加速、逐位一致 $\max|d| \leq 1e-15$ (`AGENTS.md:64`)。

5.10 A10 综合分析

与整体联系: 该链正是第 2/3 节核心层的主轴; 对象映射上, gauge 链接 $\leftrightarrow$ 规范场、 M 矩阵 $\leftrightarrow$ 非定域胶子算符色迹、 O 组合 $\leftrightarrow$ 横向极化投影的胶子 TMD 算符。深层原因: 用梯度流做非微扰重整化, 规避算符混合与混合方案的短/长距拼接。

5.11 A11 结尾总结

本链以可回放、可测试的模块化方式实现梯度流重整化胶子 TMD-PDF，且经一致性验证与规模化实战双重确认（确证）。

5.12 A12 结尾评价

优点：后端解耦、h5 可移植、测试保护网强。可改进：cpp 后端仅占位；连续极限外推仅单系综维度，未做 $a/P_z/m_\pi/L$ 联合外推的系统性误差量化（见局限）。

5.13 A13 补充说明

假设/局限：物理结论（pn 2pt=0、meff $\approx$ 1.12 GeV）属「已验证结论」不可改（AGENTS.md:138）；单组态尺度设定 t_0 依赖 lattice 常数。

5.14 A14 参考源

本步证据见第 7 节参考源清单（_gradient_flow.py/_tmd.py/_matching.py/AGENTS.md 等）。

5.15 A15 附录与附件

原始日志见 logs/test0/、logs/stab1/、logs/test6/ 各 run-local 输出。

6 关键代码/文档片段

6.1 Wilson flow 单步 (RK3)

```

1 def wilson_flow_step(U, eps):
2     """单步 RK3 (Luescher 2010 Eq.(3.8)) :  $V(t+\epsilon) = \exp(\sum c_i Z_i) \cdot V(t)$ 。"""
3     cp = get_backend()
4     W0 = U
5     Z0 = flow_derivative(W0)
6     W1 = su3_exp(0.25 * eps * Z0) @ W0
7     Z1 = flow_derivative(W1)
8     W2 = su3_exp((8.0 / 9.0) * eps * Z1 - (17.0 / 36.0) * eps * Z0) @ W1
9     Z2 = flow_derivative(W2)
10    return su3_exp((3.0 / 4.0) * eps * Z2 - (8.0 / 9.0) * eps * Z1
11                + (17.0 / 36.0) * eps * Z0) @ W2

```

6.2 胶子 TMD 算符 O 组合

```

1 def gluon_tmd_operator(U, z, b_perp, z_dir=2, b_dir=0, L=None):
2     """可乘性重整化组合  $O(z, b_\perp) = M^{\{tx;tx\}} + M^{\{ty;ty\}} - 2M^{\{xy;xy\}}$  (Eq.:0_mult_tmd) 。
3
4     Returns:

```

```

5      逐格点 0 值, 形状 (Nt,Nz,Ny,Nx)。
6      """
7      M_ttxt = M_mu_lambda_nu_rho(U, 0, 1, 0, 1, z, b_perp, z_dir, b_dir, L)
8      M_tyty = M_mu_lambda_nu_rho(U, 0, 2, 0, 2, z, b_perp, z_dir, b_dir, L)
9      M_xyxy = M_mu_lambda_nu_rho(U, 1, 2, 1, 2, z, b_perp, z_dir, b_dir, L)
10     return M_ttxt + M_tyty - 2.0 * M_xyxy

```

6.3 Clover 场强四叶构造 (节选)

```

1  def plaquette_clover(g, mu: int, nu: int):
2      """F_{μν} = -i/8 Σ_k (P_k - P_k†), 四叶 Clover 平均 (donghx)。
3
4      Args:
5          g: gauge, 形状 (Nt,Nz,Ny,Nx,4,3,3), CPU(ndarray) 或 GPU(cupy)。
6          mu, nu: Lorentz 指标 (0=t, 1=z, 2=y, 3=x), mu != nu。
7      Returns:
8          F_{μν}, 形状 (Nt,Nz,Ny,Nx,3,3)。
9      """
10     cp = get_backend()
11     m = cp.matmul
12     a_mu = 3 - mu # 空间轴
13     a_nu = 3 - nu
14
15     g_lu = cp.roll(g, 1, axis=a_mu)
16     g_rd = cp.roll(g, 1, axis=a_nu)
17     g_ld = cp.roll(g_lu, 1, axis=a_nu)
18
19     tr = (0, 1, 2, 3, 5, 4) # 共轭转置 (色指标 Hermitian 共轭)
20
21     # P1 = P_{μν} (einsum→cuBLAS matmul 等价改写, 逐位一致)
22     p1 = m(g[..., mu, :, :],
23            cp.roll(g, -1, axis=a_mu)[..., nu, :, :])
24     p1 = m(p1, cp.roll(g, -1, axis=a_nu)[..., mu, :, :].conj()
25            .transpose(*tr))
26     p1 = m(p1, g[..., nu, :, :].conj().transpose(*tr))
27
28     # P2 = P_{v,-μ}
29     p2 = m(cp.roll(g_lu, -1, axis=a_mu)[..., nu, :, :],
30            cp.roll(g_lu, -1, axis=a_nu)[..., mu, :, :].conj()
31            .transpose(*tr))
32     p2 = m(p2, g_lu[..., nu, :, :].conj().transpose(*tr))
33     p2 = m(p2, g_lu[..., mu, :, :])
34
35     # P3 = P_{-μ,-ν}
36     p3 = m(cp.roll(g_ld, -1, axis=a_nu)[..., mu, :, :].conj()
37            .transpose(*tr),
38            g_ld[..., nu, :, :].conj().transpose(*tr))

```

6.4 NLO 匹配核 g_0..g_3 (按 ξ 分区)

```

1  def _matching_kernels(cxi, y_matr, Pz_GeV, mu_, lambda_s):
2      """按 ξ=x/y 分区计算匹配核 g_xy (g_0 + 对应分区的 g_1/g_2/g_3)。"""

```



```

3  def g_1(xi_, y_):
4      return (-2.0 * (1.0 - xi_ + xi_ ** 2.0) ** 2.0 / (1.0 - xi_)
5              * np.log(xi_ / (xi_ - 1.0))
6              - (11.0 - 28.0 * xi_ + 18.0 * xi_ ** 2.0 - 12.0 * xi_ ** 3.0)
7              / (6.0 * (1.0 - xi_)))
8
9  def g_2(xi_, y_):
10     res = (2.0 * (1.0 - xi_ + xi_ ** 2.0) ** 2.0 / (1.0 - xi_)
11            * (-np.log(mu_ ** 2.0 / (4.0 * y_ ** 2.0 * Pz_GeV ** 2.0))
12              + np.log(xi_ * (1.0 - xi_))))
13     res -= ((15.0 - 56.0 * xi_ + 102.0 * xi_ ** 2.0
14             - 96.0 * xi_ ** 3.0 + 48.0 * xi_ ** 4.0)
15            / (6.0 * (1.0 - xi_)))
16     return res
17
18 def g_3(xi_, y_):
19     return (2.0 * (1.0 - xi_ + xi_ ** 2.0) ** 2.0 / (1.0 - xi_)
20            * np.log(xi_ / (xi_ - 1.0))
21            + (11.0 - 28.0 * xi_ + 18.0 * xi_ ** 2.0 - 12.0 * xi_ ** 3.0)
22            / (6.0 * (1.0 - xi_)))
23
24 def g_0(xi_, y_):
25     return (5.0 / 6.0 * (-1.0 / np.abs(1.0 - xi_)
26                        + 2.0 * Si(((1.0 - xi_) * np.abs(y_) * lambda_s))
27                        / (pi * (1.0 - xi_))))
28
29 mask_1 = (cxi < 0)
30 mask_2 = (0 < cxi) & (cxi < 1)
31 mask_3 = (cxi > 1)
32
33 g_xy = np.zeros_like(cxi)
34 g_xy[mask_1] = -g_0(cxi[mask_1], y_matr[mask_1]) - g_1(cxi[mask_1], y_matr[mask_1])
35 g_xy[mask_2] = g_0(cxi[mask_2], y_matr[mask_2]) + g_2(cxi[mask_2], y_matr[mask_2])
36 g_xy[mask_3] = g_0(cxi[mask_3], y_matr[mask_3]) + g_3(cxi[mask_3], y_matr[mask_3])
37 return g_xy

```

6.5 匹配 Z_{ij} 矩阵结构

```

1  g_xy = _matching_kernels(cxi, y_matr, Pz_GeV, mu_, lambda_s)
2  g_ij = g_xy
3
4  delta_ij = np.eye(n_len)
5  dx_matr = np.diag(xx / np.abs(xx)) * dx
6
7  c_alp_lo = dx_matr * alpha_s * CA / 2.0 / pi
8  m_alp_lo = g_ij / y_matr - delta_ij * y_matr * np.sum(
9      g_ij / (y_matr ** 2.0), axis=1)
10 z_ij = delta_ij + c_alp_lo @ m_alp_lo
11
12 return np.linalg.inv(z_ij) @ hR_tilde_data

```

7 参考源清单

参考源	类型	内容/作用
AGENTS.md:10	仓库	全量测试 confest.py (18 项)
AGENTS.md:11	仓库	verify_consistency.py(A-E 全 0 差异)
AGENTS.md:70-78	仓库	蒸馏管线 9 步、237/237 PASS、rel=0
AGENTS.md:105-110	仓库	stab1 物理断言 P0 meff≈1.12、P2≈1.56
AGENTS.md:138	仓库	反模式：不修改已验证物理结论
pyqcd/AGENTS.md	仓库	主包子包结构与来源表
pyqcd/renorm/AGENTS.md	仓库	renorm 子包权威说明
pyqcd/renorm/_gradient_flow.py:125-135	仓库	wilson_flow_step RK3
pyqcd/renorm/_gradient_flow.py:163-173	仓库	flow_action_density E(t)
pyqcd/operator/_gluon_ope.py:63-115	仓库	plaquette_clover 四叶平均
pyqcd/renorm/_tmd.py:152-161	仓库	$O = M^{tx};tx + M^{ty};ty - 2M^{xy};xy$
pyqcd/renorm/_tmd.py:203-213	仓库	self_renormalized_ratio
pyqcd/renorm/_zr.py:84-135	仓库	th_ZR / fit_ZR 重整化
pyqcd/renorm/_hybrid.py:59-119	仓库	hR_lambda 混合方案外推
pyqcd/renorm/_matching.py:25-61	仓库	g_0..g_3 匹配核
pyqcd/renorm/_matching.py:94-105	仓库	hR_PDF Z_ij 反演
pyqcd/renorm/_tmdextract.py:116	仓库	Z_ij 数值结构说明
pyqcd/tools/_torch_backend.py:222-254	仓库	torch.Tensor 补丁 (transpose/astype-/repeat)
pyqcd/pipeline/_steps.py:1418	仓库	run_pipeline 9 步调度
refer/	参考	只读参考库 (zengch/donghx/huangcl/git-rep/*) , 逻辑照抄不 import

8 结论与局限

结论

结构: pyqcd 主包以 renorm/operator/analysis/pipeline 为核心层, tools/parallel 为工程支撑; 关系: gauge→flow→算符→重整化→匹配→外推的数据依赖链清晰解耦; 思路: 照抄成功实例 + 自包含重写 + 测试保护网, 使物理可复现且可扩展; 主题结论: 梯度流重整化胶子 TMD-PDF 计算链完整实现并经一致性验证 (237/237 PASS, A-E 0 差异)。

参考背景

refer/ 下 zengch/donghx/huangcl 等提供算法原型与物理文档，PyQUDA/LQCD_Master 等提供上游实现参照；其 analy/pure 产物（20260817）已归档于各自 docs/，本主库报告与之互补、不重复。

局限与风险

未覆盖项：(1) cpp/ 后端仅占位未实现，未评估；(2) refer/ 内部 4+ 个外来库仅作参考引入、未深剖；(3) 连续极限外推 `_extrapolate.py` 仅给出拟合入口，未做 $a/Pz/m_\pi/L$ 联合外推的系统性误差量化，标注 **未验证**；(4) 部分 runtime 数值（峰值显存/耗时）来自 AGENTS.md 实测记录，未独立重测。